



Assignment 02

Subject: Object-Oriented Programming
Total Marks: 40

Post Date: 30-April-2026
Due Date: 18-May-2026

**Course
Instructor**

Talha Shahid

Please submit the link to your **public GitHub repository** for Assignment 2. Ensure that your repository is properly structured, clearly labeled, and accessible.

Your repository must include:

- All relevant source code files (including header files, if applicable)
- A well-organized directory structure
- A **Markdown (.md) documentation file** (e.g., README.md or docs.md) containing:
 - Website screenshots.
 - Relevant code snippets for the sections replicated from the website.
 - Clear reasoning and explanation of your implementation.

It is recommended to name your repository or structure it in a way that clearly identifies your submission (e.g., **A2-[StudentID]**).

Important Instructions:

- Submissions made after the deadline will not be accepted and will receive zero marks.
- Assessment will be based on both **code quality** and **documentation**, each carrying equal weightage (50% each).
- Marks will also reflect the **originality and uniqueness** of your work.
- Any form of **plagiarism, copying, or cheating** will result in a zero grade.

Only GitHub repository links will be accepted. Ensure that your repository is public before submission.



Assignment:

Extend your **Assignment 01** by applying advanced OOP concepts such as **Inheritance**, **Polymorphism**, **Abstraction**, **Operator Overloading**, and **Friend functions** to improve design and flexibility.

Requirements

- 1. Inheritance** **08 Marks**
 - Create base and derived classes such as, **User-> Buyer, Seller, Admin** and **Vehicle->Car, Bike**.
 - Implement at least **5** to **6** Inheritance relationships and justify them.
- 2. Polymorphism** **05 Marks**
 - Use function overloading and function overriding (**virtual** functions).
 - Example: **displayDetails()** behaving differently for each vehicle type.
- 3. Abstraction** **10 Marks**
 - Create at least **3** to **4** Abstract classes with pure **virtual** functions.
 - Hide implementation details and expose only essential features.
 - Create separate header (**.h**) files for each abstract class to define the interfaces, and implement their functionality in corresponding source (**.cpp**) files.
- 4. Operator Overloading** **07 Marks**
 - Overload operators such as **=** and **+** to compare or combine objects (at least **3** to **4**).
 - Provide proper reasoning for usage.
- 5. Friend Functions** **05 Marks**
 - Use **Friend** function to access **private** data where needed(at least **3** to **4**).
 - Justify why it is required.
- 6. Additional Requirements** **05 Marks**
 - Properly integrate the instructions with **Assignment 01**.
 - Update **class diagrams** and provide **clear comments** explaining each concept.